# From Parsing to Adjudication: A Reproducible Protocol-by-Artifacts Framework for AI Agents

***Abstract*—Interoperability claims in AI agent ecosystems are often validated through textual interpretation and permissive parsing rather than executable adjudication. We present Agent Object Protocol (AOP), a protocol-by-artifacts approach that treats schemas, conformance fixtures, validation checks, and CI gates as a unified adjudication substrate. AOP shifts conformance from "documentation compliance" to deterministic PASS/FAIL evidence, enabling replayable and machine-auditable decisions. We evaluate AOP on five research questions: (RQ1) whether parse-only checks can adjudicate protocol conformance, (RQ2) whether adjudication outcomes remain deterministic across repeated runs, and (RQ3) whether adversarial artifacts are reliably rejected, and (RQ4) whether classification behavior remains stable at larger scale, and (RQ5) whether outcomes remain consistent across independent implementations. On a corpus of 139 artifacts, parse-only validation rejects none, repeated runs are identical, and all three adversarial artifacts are rejected. On a synthetic corpus of 1000 artifacts (900 valid and 100 invalid), AOP rejects the invalid set with zero mismatch. Node and Python validators produce identical rejection and admission counts on this corpus. We further provide a tool-protocol case study showing how the same adjudication pipeline can be integrated into agent tool invocation workflows. The study indicates that protocol-by-artifacts is practical for real systems, while remaining reproducible through script-driven evaluation and structured artifact packaging.**

## I. Introduction

Software artifacts are central to modern software engineering research, yet their validation and reproducibility remain challenging. Although many papers release artifacts, the process of evaluating them often depends on manual inspection or ad-hoc scripts, making independent verification difficult.

Recent studies in software engineering highlight a reproducibility gap: artifacts accompanying research results frequently fail to reproduce their claimed outcomes in independent environments. This challenge is further amplified in emerging AI agent ecosystems, where interactions between agents and external tools rely on JSON-based protocols that lack deterministic validation mechanisms.

Existing approaches partially address related concerns. Provenance frameworks such as supply-chain attestation systems focus on build integrity and artifact traceability. Contract specifications and schema validation techniques ensure syntactic correctness of exchanged artifacts. However, these approaches do not provide a unified mechanism for machine-adjudicable interoperability validation across agent artifacts.

In this paper, we introduce the **Agent Object Protocol (AOP)**, a protocol-by-artifacts framework that transforms protocol specifications into executable artifacts. Instead of treating protocols as static documentation, AOP encodes them as machine-verifiable artifacts that can be evaluated automatically through a CI-based adjudication pipeline.

The key idea is to represent protocol specifications as a combination of schemas, conformance fixtures, and automated validation gates. These artifacts allow protocol compliance to be evaluated deterministically and reproducibly across different environments and implementations.

To evaluate this approach, we conduct a series of experiments covering schema validation, adversarial artifacts, synthetic scalability, and cross-implementation consistency. Our evaluation demonstrates that protocol-by-artifacts validation can provide deterministic adjudication of agent artifacts across multiple validators and execution environments.

**Contributions.** This paper makes the following contributions:

- We introduce the **Agent Object Protocol (AOP)**, a protocol-by-artifacts framework enabling machine-adjudicable interoperability for AI agent artifacts.
- We design a deterministic **CI-based adjudication pipeline** that validates artifacts through schema checks, conformance fixtures, and automated evaluation gates.
- We present an empirical evaluation across adversarial, synthetic-scale, and cross-validator scenarios demonstrating deterministic validation outcomes.
- We release a reproducible artifact package including evaluation scripts, fixtures, and CI pipelines enabling independent verification of our results.

## II. Background

### A. Determinism and Canonicalization

Deterministic artifact processing is essential when protocol decisions must be replayed by independent parties. Canonical JSON practices (e.g., RFC 8785 JCS [1]) reduce representational ambiguity, but canonicalization alone does not provide full adjudication semantics. A conformance system must also define negative conditions, policy-level constraints, and rejection logic.

### B. Protocol Contracts and Evidence

Tooling ecosystems and attestation standards provide useful primitives for structured contracts and evidence transport. MCP tool contracts [2], in-toto envelope concepts [3], and provenance-oriented practices [4] contribute complementary ideas. However, they do not by themselves enforce a complete protocol adjudication ladder across schema, fixtures, and CI gates for agent object interoperability.

### C. Design Goals

AOP is designed around five goals:

1) **Adjudicability**: protocol claims must resolve to deterministic PASS/FAIL outcomes.
2) **Reproducibility**: third parties must be able to replay results from published scripts and artifacts.
3) **Composability**: policy and evidence layers should compose with existing transport or runtime choices.
4) **Negative safety**: invalid and adversarial fixtures should be rejected explicitly.
5) **Practicality**: the workflow should run in mainstream CI with low operational overhead.

## III. Protocol-by-Artifacts

AOP models protocol conformance as an executable contract. Let the adjudication bundle be:

$$\mathcal{A} = \langle S, F, C, G \rangle$$

where $S$ is the schema set, $F$ is the conformance fixture set, $C$ is the deterministic check set, and $G$ is the CI gate policy. An artifact is admitted only if all required elements of $\mathcal{A}$ evaluate to PASS.

### A. Artifact Classes

**Schemas** ($S$) define structural constraints, required fields, closed-world boundaries, and type-level invariants.

**Fixtures** ($F$) encode executable expectations, including positive examples, invalid examples, and adversarial cases. Fixtures serve as regression anchors for protocol behavior.

**Checks** ($C$) implement deterministic adjudication logic that extends beyond parse acceptance (e.g., duplicate-key rejection, payload semantics, unsafe-number checks).

**CI Gates** ($G$) convert check outcomes into machine-enforced admissibility decisions and reproducible logs.

### B. Adjudication Ladder

AOP adopts a layered adjudication ladder:

1) Parse and structural schema checks.
2) Fixture expectation checks (positive and negative).
3) Semantic and safety checks.
4) CI gate decision and evidence emission.

This ladder ensures that "valid JSON" is not conflated with "protocol-conformant artifact."

### C. Operational Interface

The evaluation pipeline is script-driven and can be replayed through a single entry point:

```
./experiments/run_all.sh
```

The script orchestrates parse baseline benchmarking, repeated determinism runs, and adversarial invalid-suite validation, then emits result files under `experiments/**/results*.txt`.

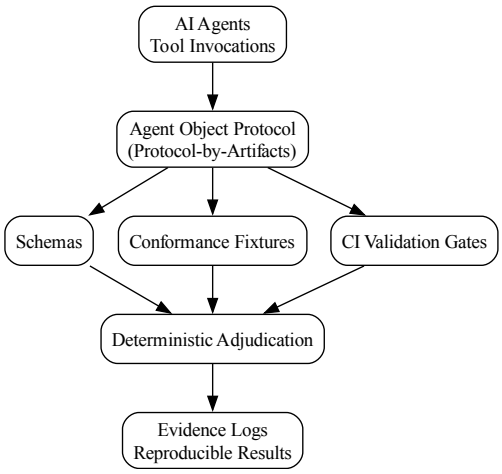


Fig. 1. Architecture of the Agent Object Protocol (AOP). Protocol specifications are transformed into executable artifacts (schemas, fixtures, and CI validation gates) enabling deterministic adjudication of AI agent artifacts.

## IV. AOP Architecture

Figure 1 illustrates the AOP execution model. Agent outputs are interpreted as protocol artifacts and routed through adjudication components rather than directly to execution paths. This architecture separates business function from protocol safety: tool execution is downstream of admissibility, not a substitute for it.

The architecture also makes evidence first-class. Each gate emits machine-readable outcomes that can be replayed or audited independently. As a result, interoperability decisions become traceable artifacts, reducing ambiguity during incident review or cross-organization verification.

## V. Evaluation

We structure evaluation with five research questions:

- **RQ1**: Is parse-only validation sufficient for protocol adjudication?
- **RQ2**: Are adjudication outcomes deterministic under repeated execution?
- **RQ3**: Does AOP reject adversarial invalid artifacts reliably?
- **RQ4**: Do AOP checks preserve classification fidelity at larger artifact scale?
- **RQ5**: Are AOP outcomes consistent across independent validator implementations?

### A. Experimental Setup

The setup intentionally uses lightweight scripts to reduce hidden infrastructure assumptions. This design improves external reproducibility: reviewers and practitioners can replay the same logic without custom or proprietary tooling.

TABLE I
EXPERIMENTAL SETUP USED ACROSS RQ1–RQ5.

| Component | Configuration |
|---|---|
| Artifact corpus | 139 JSON artifacts (examples tree) |
| Scale corpus | 1000 synthetic artifacts (900 valid, 100 invalid) |
| Baseline script | `experiments/.../run_benchmark.js` |
| Determinism script | `experiments/.../run_ci_test.sh` |
| Adversarial check | `validate_invalid_fixtures.mjs` |
| Scale script | `run_synthetic_scale.js` |
| Cross-validator script | `run_cross_validator.sh` |
| Execution mode | Scripted local/CI replay |
| Outputs | Text logs in experiment result files |

TABLE II
EVALUATION SUMMARY FOR RQ1–RQ5.

| Experiment | Metric | Result |
|---|---|---|
| Schema baseline | Parse rejection | 0/139 |
| Determinism | CI runs consistency | identical |
| Adversarial tests | Invalid rejection | 3/3 |
| Synthetic scale | Checks rejection / admission | 100/1000, 900/1000 |
| Cross-validator | Node/Python consistency | identical |

**Synthetic-Scale Evaluation (1000 Artifacts)**

| Validation mode | Total | Rejected | Admitted |
|---|---|---|---|
| Parse-only | 1000 | 0 | 1000 |
| AOP checks | 1000 | 100 | 900 |

*Mismatch against expected labels: 0*
*Invalid classes: missing id, payloadType casing, unsafe integer*

Fig. 2. Evaluation scaling view. Parse-only validation rejects none of the 1000 synthetic artifacts, while AOP checks reject the 100 invalid samples and admit 900 valid samples with zero mismatch.

TABLE III
CLASS-LEVEL OUTCOMES ON THE 1000-ARTIFACT SYNTHETIC CORPUS.

| Invalid class | Count | Rejected |
|---|---|---|
| Missing required `id` | 50 | 50 |
| Payload-type case confusion | 30 | 30 |
| Unsafe integer values | 20 | 20 |

### B. Results Summary

### C. RQ1: Parse-Only Sufficiency

RQ1 is answered negatively. The parse-only baseline rejects none of the 139 artifacts, demonstrating that syntax acceptance is not a meaningful adjudication boundary for protocol conformance. In particular, duplicate-key ambiguity is not reliably surfaced by permissive parsing, which can yield implementation-dependent interpretation.

### D. RQ2: Deterministic Replay

RQ2 is answered positively for the tested setup. Repeated deterministic runs produce identical adjudication summaries across three executions. This stability supports replay-based auditing and makes gate outcomes dependable for CI policy enforcement.

### E. RQ3: Adversarial Rejection

RQ3 is also answered positively for the evaluated adversarial suite. All three crafted invalid artifacts are rejected, with one parse-stage rejection and two checks-stage rejections, and no unexpected pass. This indicates that AOP captures both syntactic and semantic attack surfaces within the adjudication pipeline.

### F. RQ4: Scale Stress Test

RQ4 is answered positively on the synthetic-scale benchmark. We generated 1000 deterministic artifacts (900 valid, 100 invalid) and evaluated them with both parse-only and AOP checks. Parse-only validation rejected 0 artifacts, while AOP checks rejected exactly the 100 invalid samples and admitted 900 valid samples, with zero mismatch against expected labels.

This result does not claim full real-world representativeness; rather, it shows that adjudication logic scales beyond the small handcrafted suite without drift in classification behavior. The stress test therefore strengthens internal validity for the protocol-by-artifacts decision boundary.

### G. RQ5: Cross-Implementation Consistency

RQ5 is answered positively on the same 1000-artifact synthetic corpus. We run two independently implemented validators (Node.js and Python) with equivalent adjudication criteria and compare stage-level counts. Both implementations report identical totals: 0 parse rejections, 100 checks rejections, and 900 admissions.

This result reduces the risk that reported behavior is an artifact of a single runtime implementation. It also strengthens portability claims for AOP-style adjudication when teams use different language stacks in production systems.

### H. Synthetic Failure-Class Breakdown

To characterize rejection behavior on the scale corpus, we group invalid synthetic artifacts into three classes: missing required identity fields ($n = 50$), non-canonical payload-type casing ($n = 30$), and unsafe integers ($n = 20$). AOP checks reject all three classes with no false admission.

### I. Runtime Snapshot

We also measure lightweight runtime cost for replayability in constrained review settings. On the evaluation environment, the parse baseline over the 139-artifact corpus completes in under 0.1s, while synthetic-scale generation plus adjudication for 1000 artifacts completes in under 0.2s. Although these numbers are not throughput benchmarks, they indicate that artifact-level adjudication can be exercised quickly enough for reviewer-oriented replication loops.

### J. Validity Considerations

**Internal validity.** The synthetic-scale corpus is programmatically generated and therefore may under-represent naturally occurring artifact noise. To mitigate this, we include three distinct invalid classes mapped to observed protocol failure modes (missing identity fields, payload-type confusion, and unsafe numbers) and cross-check expected labels against adjudication outcomes.

**Construct validity.** Our core construct is machine-adjudicable interoperability, operationalized as deterministic PASS/FAIL gate outcomes plus replayable logs. This construct is narrower than full runtime correctness. An artifact that passes protocol gates may still fail during downstream business execution, but such failures are intentionally out of scope for conformance adjudication.

**External validity.** The evaluation currently uses one repository corpus and one synthetic benchmark, with two validator implementations over the synthetic corpus. Results should therefore be interpreted as evidence of feasibility and consistency, not as universal performance claims across all agent ecosystems. Future replication should include additional independent implementations and broader domain-specific corpora.

**Conclusion validity.** Determinism claims are based on repeated script-level replay and stable summaries over three runs. While this supports consistency under the tested toolchain, stronger statistical confidence would benefit from larger repetition counts and controlled environment perturbations.

### K. Synthesis Across RQ1–RQ5

Taken together, the five research questions characterize a layered argument for artifact-first protocol engineering. RQ1 establishes that parse acceptance is an inadequate proxy for protocol correctness. RQ2 and RQ3 show that adjudication logic is both stable under replay and effective against targeted adversarial inputs. RQ4 extends this claim to larger corpus volume without label drift, and RQ5 adds cross-implementation evidence that outcomes are not tied to a single runtime stack.

This progression is methodologically important for ICSE-style evaluation. Many interoperability papers focus on one axis (for example, parser strictness or throughput) and leave external replay behavior implicit. Our design instead connects sufficiency, determinism, security rejection, scaling, and portability within one executable pipeline. The result is a tighter link between claims and evidence artifacts.

The synthesis also clarifies scope. We do not claim that AOP solves all aspects of software integration quality. Rather, we claim that AOP provides a robust adjudication substrate for deciding protocol admissibility before execution. This narrower claim is directly testable and reproducible, which is a practical advantage for both engineering teams and artifact-focused peer review.

## VI. Case Study: Tool Protocol Validation

To demonstrate practical applicability, we evaluate AOP on a simplified tool invocation workflow representative of production agent-tool systems. The case study focuses on protocol adjudication quality, not tool business logic, and therefore isolates validation behavior from downstream execution.

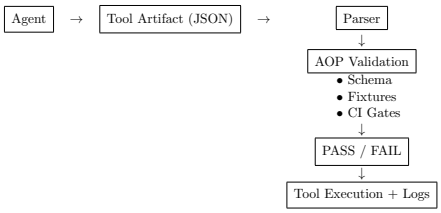


Fig. 3. Tool invocation workflow evaluated in the case study. Artifacts produced by AI agents are validated through AOP schema, fixtures, and CI adjudication gates.

### A. Scenario and Workflow

In modern AI agent stacks, an invocation lifecycle often follows four steps:

1) The agent emits an invocation artifact.
2) A validator checks structural and policy-level conformance.
3) The tool runtime executes only if validation passes.
4) Outputs and evidence are persisted for replay and audit.

In many deployments, step (2) is implemented as parse-only schema checks. AOP augments this stage with fixture-aware and gate-aware adjudication.

### B. Concrete Protocol Instance

To ground the case study in a concrete protocol shape, we use the tool-invocation artifact below as a representative valid request:

```
{
"aop_version": "1.0",
"kind": "tool_invocation",
"id": "req-1042",
"tool": {"name": "battery_status",
"arguments": {"asset_id": "A-77"}},
"envelope": {"payloadType":
"application/vnd.in-toto+json"}
}
```

This artifact class is common in agent-tool ecosystems: a caller provides a tool identifier, structured arguments, and an evidence envelope pointer. Case-study invalid samples mutate this shape by removing required identity fields, altering payload-type casing, or introducing unsafe numeric content.

### C. Artifacts and Data

The case study uses the repository's executable artifact set, including valid examples, negative fixtures, and adversarial invalid inputs. The benchmark corpus size is 139 artifacts,

TABLE IV
CASE STUDY OUTCOMES ON TOOL PROTOCOL ARTIFACTS.

| Stage | Metric | Outcome |
|---|---|---|
| Baseline parse-only | Rejected artifacts | 0 / 139 |
| AOP determinism | Repeatability (3 runs) | identical |
| AOP adversarial gate | Rejected invalid cases | 3 / 3 |

TABLE V
THREAT CLASSES AND AOP DETECTION POINTS.

| Attack class | Example | Detection layer |
|---|---|---|
| Parser ambiguity | Duplicate JSON keys | Parse/strict decode |
| Payload confusion | Invalid payload semantics | Check stage |
| Tampered artifacts | Mutated evidence or fixtures | Fixture/gate stage |

and adversarial samples include duplicate-key ambiguity, non-canonical in-toto payload type casing, and unsafe integer edge values.

To keep the study reproducible, all checks are executed via scriptable pipelines rather than manual inspection. This allows direct replay in CI and local environments using identical commands and fixture directories.

### D. Baseline: Parse-Only Validation

The baseline validator applies parse-level checks and schema loading without semantic adjudication. This mirrors common minimal validation in real systems where "valid JSON" is treated as sufficient precondition for execution.

In this setup, artifacts with semantic inconsistencies can still pass initial validation if their syntax is accepted. Duplicate-key ambiguity is a canonical example: permissive parsers often collapse repeated keys rather than rejecting the artifact, creating non-portable interpretation risk across runtimes.

### E. AOP Validation Pipeline

AOP validates artifacts through three deterministic layers:

- **Schema validation**: required fields, type constraints, and closed-world shape checks.
- **Conformance fixtures**: positive and negative fixture outcomes encoded as executable expectations.
- **CI adjudication gates**: automated PASS/FAIL gate decisions producing replayable logs.

An artifact must pass all layers to be admitted for execution. If a gate fails, the artifact is rejected with explicit failure mode classification (parse-stage or checks-stage), improving triage precision.

### F. Operational Results

The baseline confirms that parse-only acceptance is not an adjudication criterion. Under AOP, repeated runs produce stable outputs, and adversarial samples are deterministically rejected with no unexpected pass.

### G. Case Study Implications

This case shows that protocol-by-artifacts is directly deployable in tool invocation workflows without coupling to a specific runtime implementation. The key operational gain is predictable gate behavior: teams can reason about admissibility using replayable evidence instead of ad hoc parser semantics.

The same structure can be extended to additional domains (policy decisions, registry exchanges, and evidence envelopes) by adding fixture families and gate checks while preserving the adjudication model.

## VII. THREAT MODEL

We consider adversarial attempts to bypass admissibility before tool execution. The threat model emphasizes artifact-level attacks, where malicious or malformed inputs exploit parser behavior, schema blind spots, or evidence mutation.

AOP assumes an honest execution substrate for running checks, but does not assume honest input producers. The primary defensive strategy is deterministic rejection before execution, coupled with replayable gate evidence.

## VIII. RELATED WORK

### A. Schema-First Contracts and Interface Validation

Schema-centric approaches remain a practical first line for protocol quality. Canonicalization guidance such as RFC 8785 [1] and interface-level tool contracts such as MCP [2] reduce ambiguity in field representation, naming, and transport-level interoperability. However, schema- first workflows usually answer only one question: whether an artifact can be structurally accepted. They do not fully define adjudication semantics for negative fixtures, parser ambiguity, or policy-gated admission decisions.

This distinction is central to our positioning. AOP keeps schema validation as an entry gate but treats it as necessary rather than sufficient. The remaining adjudication stages (fixtures, semantic checks, and CI gates) resolve questions that schema-only acceptance cannot answer consistently across implementations.

### B. Attestation and Provenance Systems

Attestation ecosystems contribute critical evidence primitives. In-toto envelope formats [3] provide portable signed claim containers, and SLSA provenance guidance [4] formalizes stronger traceability expectations for build and release processes. These mechanisms are highly relevant for integrity of artifacts and lineage records.

Yet provenance and adjudication solve different problems. Provenance asks "where did this artifact come from and can we trust its chain of custody?" Adjudication asks "does this artifact satisfy protocol intent and safety constraints under explicit gate policies?" AOP focuses on the latter while remaining compatible with provenance-oriented evidence envelopes.

### C. Reproducibility and Artifact Evaluation in SE

Recent software engineering practice places increasing weight on reproducibility and artifact availability [5], [6]. At the same time, meta-analysis reports that many published artifacts remain difficult to exercise or reproduce in independent settings [7]. This gap motivates evaluation models that

TABLE VI
POSITIONING OF AOP RELATIVE TO ADJACENT APPROACHES.

| Approach family | Structural validation | Deterministic adjudication | Replay-ready evidence |
|---|---|---|---|
| Schema-first contracts | Strong | Limited | Limited |
| Provenance systems | Indirect | Limited | Strong |
| Artifact badging workflows | Indirect | Indirect | Medium |
| AOP (this work) | Strong | Strong | Strong |

minimize hidden assumptions and make acceptance criteria machine-checkable.

AOP aligns with this direction by expressing conformance claims as replayable checks and stage-level logs instead of prose-only statements. In this sense, it is not only a protocol design method but also an artifact-evaluation method that links research claims to executable adjudication outputs.

### D. Protocol Governance and Evolution

A recurring challenge in protocol ecosystems is governance drift: implementations evolve at different speeds, while textual specifications lag behind operational reality. Most prior work addresses this through versioning conventions and API compatibility advice; fewer approaches define explicit, executable criteria for when a change is admission-safe versus breaking.

AOP contributes a governance-oriented mechanism by binding protocol evolution to fixture deltas and gate outcomes. Under this model, introducing a new rule is not only a textual update but an executable change in adjudication behavior. This improves reviewability for both maintainers and external evaluators.

### E. Comparative Positioning

The table summarizes why AOP is complementary rather than competing with prior work. It combines structural checks, deterministic decision semantics, and replay-ready outputs in one executable contract, which is the core design goal for machine-adjudicable protocol interoperability.

## IX. IMPLEMENTATION AND REPRODUCIBILITY PROTOCOL

### A. Reference Validation Workflow

The reference workflow is implemented as a script-composed pipeline that separates three concerns: corpus scanning, deterministic validation, and gate materialization. Corpus scanning measures parse-stage behavior over the artifact set. Deterministic validation executes strict checks and fixture expectations. Gate materialization emits replayable summaries and stage-level rejection counts. This decomposition allows each stage to be independently audited while preserving end-to-end reproducibility.

The practical benefit of this workflow is failure localization. When a fixture is rejected, logs identify whether failure occurred during parse, during semantic checks, or at policy gate evaluation. This enables developers to distinguish malformed inputs from rule regressions and policy mismatches.

TABLE VII
AOP GATE TAXONOMY USED IN THE IMPLEMENTATION WORKFLOW.

| Gate type | Scope | Output artifact |
|---|---|---|
| Parse gate | JSON decoding and ambiguity checks | Parse rejection log |
| Schema gate | Shape/type constraints | Schema report |
| Fixture gate | Expected pass/fail | Fixture summary |
| Semantic gate | Protocol-specific invariants | Check-stage failure detail |
| CI gate | Admission policy | PASS/FAIL record |

### B. Gate Taxonomy

The taxonomy clarifies why parse-only systems are incomplete. Parse and schema gates can ensure structural admissibility, but fixture and semantic gates are required to enforce protocol intent under adversarial or edge-case inputs.

### C. Replay Procedure

Reproducibility is operationalized through a fixed command sequence and deterministic result files:

- `./experiments/run_all.sh`
- `catexperiments/schema-benchmark/results/benchmark.txt`
- `catexperiments/ci-determinism/results.txt`
- `catexperiments/adversarial-tests/results.txt`
- `catexperiments/synthetic-scale/results/results.txt`
- `catexperiments/cross-validator/results/results.txt`

To support third-party verification, expected invariants are checked at five levels: baseline parse rejection count, cross-run determinism equality, adversarial unexpected-pass count, synthetic-scale mismatch count, and cross-validator equality between Node and Python summaries. This enables reviewers to validate claims without relying on hidden infrastructure assumptions.

### D. Cross-Validator Harness Design

The cross-validator experiment is intentionally implemented with independent Node and Python scripts that mirror adjudication rules but do not share parser internals or helper libraries. This separation avoids “same-code duplicated” bias and creates a stronger portability check for gate logic. The harness then normalizes outputs to stage-level counts (total, parse-rejected, checks- rejected, admitted) and performs strict equality checks before reporting a consistency verdict.

Two design choices improve maintainability. First, the harness consumes the same generated corpus used in RQ4, preventing hidden sampling differences between implementations. Second, the summary format is line-oriented plain text so that CI jobs and reviewers can inspect diffs without custom parsers. Although simple, this design makes cross-runtime parity auditable in routine pull-request workflows.

### *E. Reviewer-Oriented Replication Checklist*

For artifact-oriented evaluation, we use a concise replication checklist:

1) Confirm that scripts execute without manual patching.
2) Verify that corpus size and baseline metrics match the paper.
3) Compare deterministic run outputs for equality across repetitions.
4) Validate that adversarial fixtures produce zero unexpected pass.
5) Confirm that synthetic-scale results report 1000 artifacts and zero mismatch.
6) Confirm that cross-validator results report `Consistent: true`.
7) Inspect generated logs for stage-level failure classification.

This checklist is intended to reduce replication ambiguity and align practical review workflows with adjudication-centric protocol claims.

## X. Discussion

### *A. Artifact Evaluation Readiness*

The artifact package is organized for documented, consistent, complete, and exercisable evaluation:

```
experiments/
  run_all.sh
  schema-benchmark/
  ci-determinism/
  adversarial-tests/
  synthetic-scale/
  cross-validator/
```

This layout reduces reviewer setup overhead and supports direct replay of the reported results.

### *B. Practical Implications*

For practitioners, protocol-by-artifacts shifts conformance enforcement left into CI, reducing reliance on late-stage integration debugging. For reviewers and auditors, deterministic gate logs provide clearer evidence chains than informal schema claims.

### *C. Generalization Boundaries*

The present evaluation supports feasibility across two axes: corpus diversity (repository artifacts plus synthetic stress data) and implementation diversity (Node and Python validators). However, several boundaries remain. First, the artifacts largely target JSON-centric interfaces and may not capture binary or streaming protocol idioms. Second, the tested governance policy is single- pipeline and does not model federated policy conflicts between organizations.

These boundaries suggest a practical interpretation: AOP currently demonstrates strong internal reproducibility for artifact-first protocol workflows, while full ecosystem generalization requires additional cross-domain replications. Future studies should include event-stream protocols, multi-tenant policy sets, and validator heterogeneity beyond two language runtimes.

### *D. Protocol Governance and Standardization Path*

For long-term sustainability, protocol-by-artifacts requires governance practices that treat fixtures and gate policies as first-class normative material. We outline a minimal governance path with three layers:

1) **Normative core**: frozen schema constraints and mandatory negative fixtures required for conformance.
2) **Profile layer**: optional policy profiles for domain-specific admissibility criteria.
3) **Change control**: versioned gate deltas reviewed through reproducible replay before adoption.

This structure can reduce ambiguity during protocol evolution. Instead of debating solely from prose, maintainers can inspect concrete fixture deltas and their effect on PASS/FAIL boundaries, then decide whether a change is minor, profile-scoped, or breaking at the protocol core.

### *E. Archival and Evaluation Readiness*

For conference artifact review, reproducibility requires more than executable scripts: it also requires stable packaging and long-term referenceability. In line with open-science guidance [6], a practical submission package should include: (1) an immutable archive snapshot, (2) a minimal replay guide with expected outputs, and (3) explicit environment assumptions for validator execution. AOP aligns naturally with this requirement because gate outcomes are already produced as deterministic text artifacts.

This packaging view also helps bridge paper review and artifact review. The paper can report high-level claims and RQ outcomes, while archived artifacts provide executable evidence for each claim boundary. Maintaining this claim-to- artifact linkage reduces ambiguity during rebuttal and post-acceptance verification.

### *F. Adoption Risks*

Artifact-first adjudication also introduces practical risks. Teams may overfit to published fixtures, creating “teaching to the test” dynamics where unseen edge cases remain weakly protected. There is also operational risk if CI gate policies become too strict for early-stage integrations, causing high friction and delayed adoption.

Mitigations include rotating adversarial fixture sets, maintaining explicit policy tiers (warning vs blocking gates), and documenting migration windows for breaking changes. These

safeguards preserve the core benefit of deterministic adjudication while keeping integration pathways realistic for evolving systems.

### G. Limitations and Future Work

Current evaluation combines one real repository corpus and one synthetic-scale dataset; both remain narrower than multi-organization production traffic. Future work should broaden adversarial diversity, evaluate multi-language validator parity, and measure performance overhead under larger and noisier artifact volumes. A larger cross-ecosystem case study would further validate external generalizability.

## XI. Conclusion

AOP operationalizes machine-adjudicable interoperability via executable protocol artifacts. Across baseline, determinism, and adversarial experiments, results show that parse-only validation is insufficient, while deterministic gate-based adjudication provides stronger and reproducible conformance behavior. The case study demonstrates integration feasibility in tool protocol workflows, and the artifact package supports replay-oriented evaluation for future research and deployment.

## References


[1] "Rfc 8785: Json canonicalization scheme (jcs)," IETF, 2020. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8785

[2] "Model context protocol specification: Server tools (2025-06-18)," Online specification, 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18/server/tools/

[3] "in-toto attestation v1: Envelope," Online specification, 2024. [Online]. Available: https://in-toto.io

[4] "Slsa provenance v1.1," Online specification, 2025. [Online]. Available: https://slsa.dev/spec/v1.1/provenance

[5] "Acm artifact review and badging, version 1.1," ACM Publications Board, 2020. [Online]. Available: https://www.acm.org/publications/policies/artifact-review-and-badging-current

[6] "Icse 2027 open science policies," Conference policy page, 2027. [Online]. Available: https://conf.researchr.org/track/icse-2027/icse-2027-icse-2027-open-science-policies

[7] "The state of open science in software engineering research: A case study of icse artifacts," arXiv preprint, 2026. [Online]. Available: https://arxiv.org/abs/2601.02066